

Coping 函数（包括 *copy* 构造函数和 *copy assignment* 操作符）有可能被编译器自动创建出来，因此除非编译器所生版本做了你想要做的事（条款 5 提过其缺省行为），否则你得自己编写它们。某些情况下你或许也想支持这些函数的一般版本，这样的版本描述于条款 45。

请记住

- 复制 RAII 对象必须一并复制它所管理的资源，所以资源的 *copying* 行为决定 RAII 对象的 *copying* 行为。
- 普遍而常见的 RAII class *copying* 行为是：抑制 *copying*、施行引用计数法（reference counting）。不过其他行为也都可能被实现。

条款 15: 在资源管理类中提供对原始资源的访问

Provide access to raw resources in resource-managing classes.

资源管理类（resource-managing classes）很棒。它们是你对抗资源泄漏的堡垒。排除此等泄漏是良好设计系统的根本性质。在一个完美世界中你将倚赖这样的 classes 来处理和资源之间的所有互动，而不是玷污双手直接处理原始资源（raw resources）。但这个世界并不完美。许多 APIs 直接指涉资源，所以除非你发誓（这其实是一种少有实际价值的举动）永不录用这样的 APIs，否则只得绕过资源管理对象（resource-managing objects）直接访问原始资源（raw resources）。

举个例子，条款 13 导入一个观念：使用智能指针如 `auto_ptr` 或 `tr1::shared_ptr` 保存 **factory** 函数如 `createInvestment` 的调用结果：

```
std::tr1::shared_ptr<Investment> pInv(createInvestment()); //见条款 13
```

假设你希望以某个函数处理 `Investment` 对象，像这样：

```
int daysHeld(const Investment* pi); //返回投资天数
```

你想要这么调用它：

```
int days = daysHeld(pInv); //错误!
```

却通不过编译，因为 daysHeld 需要的是 Investment* 指针，你传给它的却是个类型为 tr1::shared_ptr<Investment> 的对象。

这时候你需要一个函数可将 RAII class 对象（本例为 tr1::shared_ptr）转换为其所内含之原始资源（本例为底部之 Investment*）。有两个做法可以达成目标：显式转换和隐式转换。

tr1::shared_ptr 和 auto_ptr 都提供一个 get 成员函数，用来执行显式转换，也就是它会返回智能指针内部的原始指针（的复件）：

```
int days = daysHeld(pInv.get()); //很好，将 pInv 内的原始指针
                                //传给 daysHeld
```

就像（几乎）所有智能指针一样，tr1::shared_ptr 和 auto_ptr 也重载了指针取值（pointer dereferencing）操作符（operator-> 和 operator*），它们允许隐式转换至底部原始指针：

```
class Investment { //investment 继承体系的根类
public:
    bool isTaxFree() const;
    ...
};

Investment* createInvestment(); //factory 函数

std::tr1::shared_ptr<Investment> //令 tr1::shared_ptr
    pi1(createInvestment()); // 管理一笔资源。

bool taxable1 = !(pi1->isTaxFree()); //经由 operator-> 访问资源。
...
std::auto_ptr<Investment> pi2(createInvestment()); //令 auto_ptr
                                                    //管理一笔资源。

bool taxable2 = !((*pi2).isTaxFree()); //经由 operator* 访问资源。
...
```

由于有时候还是必须取得 RAII 对象内的原始资源，某些 RAII class 设计者于是联想到“将油脂涂在滑轨上”，做法是提供一个隐式转换函数。考虑下面这个用于字体的 RAII class（对 C API 而言字体是一种原生数据结构）：

```
FontHandle getFont(); //这是个 C API。为求简化暂略参数。
```

```

void releaseFont(FontHandle fh);           //来自同一组 C API
class Font {                               //RAII class
public:
    explicit Font(FontHandle fh)           //获得资源;
        : f(fh)                           //采用 pass-by-value,
    { }                                    // 因为 C API 这样做。
    ~Font() { releaseFont(f); }           //释放资源
private:
    FontHandle f;                          //原始 (raw) 字体资源
};

```

假设有大量与字体相关的 C API，它们处理的是 FontHandles，那么“将 Font 对象转换为 FontHandle”会是一种很频繁的需求。Font class 可为此提供一个显式转换函数，像 get 那样：

```

class Font {
public:
    ...
    FontHandle get() const { return f; }    //显式转换函数
    ...
};

```

不幸的是这使得客户每当想要使用 API 时就必须调用 get：

```

void changeFontSize(FontHandle f, int newSize);    //C API
Font f(getFont());
int newFontSize;
...
changeFontSize(f.get(), newFontSize);    //明白地将 Font 转换为 FontHandle

```

某些程序员可能会认为，如此这般地到处要求显式转换，足以使人们倒尽胃口，不再愿意使用这个 class，从而增加了泄漏字体的可能性，而 Font class 的主要设计目的就是为了防止资源（字体）泄漏。

另一个办法是令 Font 提供隐式转换函数，转型为 FontHandle：

```

class Font {
public:
    ...
    operator FontHandle() const           //隐式转换函数
    { return f; }
    ...
};

```

这使得客户调用 C API 时比较轻松且自然：

```
Font f(getFont());  
int newFontSize;  
...  
changeFontSize(f, newFontSize);           //将 Font 隐式转换为 FontHandle
```

但是这个隐式转换会增加错误发生机会。例如客户可能会在需要 Font 时意外创建一个 FontHandle:

```
Font f1(getFont( ));  
...  
FontHandle f2 = f1;           //喔欧! 原意是要拷贝一个 Font 对象,  
                               //却反而将 f1 隐式转换为其底部的 FontHandle  
                               //然后才复制它。
```

以上程序有个 FontHandle 由 Font 对象 f1 管理,但那个 FontHandle 也可通过直接使用 f2 取得。那几乎不会有好下场。例如当 f1 被销毁,字体被释放,而 f2 因此成为“虚吊的”(dangle)。

是否该提供一个显式转换函数(例如 get 成员函数)将 RAII class 转换为其底部资源,或是应该提供隐式转换,答案主要取决于 RAII class 被设计执行的特定工作,以及它被使用的情况。最佳设计很可能是坚持条款 18 的忠告:“让接口容易被正确使用,不易被误用”。通常显式转换函数如 get 是比较受欢迎的路子,因为它将“非故意之类型转换”的可能性最小化了。然而有时候,隐式类型转换所带来的“自然用法”也会引发天秤倾斜。

你的内心也可能认为,RAII class 内的那个返回原始资源的函数,与“封装”发生矛盾。那是真的,但一般而言它谈不上是什么设计灾难。RAII classes 并不是为了封装某物而存在;它们的存在是为了确保一个特殊行为——资源释放——会发生。如果一定要,当然也可以在这基本功能之上再加一层资源封装,但那并非必要。此外也有某些 RAII classes 结合十分松散的底层资源封装,藉以获得真正的封装实现。例如 `tr1::shared_ptr` 将它的所有引用计数机构封装了起来,但还是让外界很容易访问其所内含的原始指针。就像多数设计良好的 classes 一样,它隐藏了客户不需要看的部分,但备妥客户需要的所有东西。

请记住

- APIs 往往要求访问原始资源 (raw resources)，所以每一个 RAII class 应该提供一个“取得其所管理之资源”的办法。
- 对原始资源的访问可能经由显式转换或隐式转换。一般而言显式转换比较安全，但隐式转换对客户比较方便。

条款 16: 成对使用 new 和 delete 时要采取相同形式

Use the same form in corresponding uses of new and delete.

以下动作有什么错？

```
std::string* stringArray = new std::string[100];  
...  
delete stringArray;
```

每件事看起来都井然有序。使用了 new，也搭配了对应的 delete。但还是有某样东西完全错误：你的程序行为不明确（未有定义）。最低限度，stringArray 所含的 100 个 string 对象中的 99 个不太可能被适当删除，因为它们的析构函数很可能没被调用。

当你使用 new（也就是通过 new 动态生成一个对象），有两件事发生。第一，内存被分配出来（通过名为 operator new 的函数，见条款 49 和条款 51）。第二，针对此内存会有一个（或更多）构造函数被调用。当你使用 delete，也有两件事发生：针对此内存会有一个（或更多）析构函数被调用，然后内存才被释放（通过名为 operator delete 的函数，见条款 51）。delete 的最大问题在于：即将被删除的内存之内究竟存有多少对象？这个问题的答案决定了有多少个析构函数必须被调用起来。

实际上这个问题可以更简单些：即将被删除的那个指针，所指的是单一对象或对象数组？这是个必不可少的问题，因为单一对象的内存布局一般而言不同于数组的内存布局。更明确地说，数组所用的内存通常还包括“数组大小”的记录，以便 delete 知道需要调用多少次析构函数。单一对象的内存则没有这笔记录。你可以把两种不同的内存布局想象如下，其中 n 是数组大小：